



**HERALD
COLLEGE**
KATHMANDU



UNIVERSITY OF
WOLVERHAMPTON

6CS012:Artificial Intelligence & Machine Learning Assessment

Part II: Image Classification Report

Name: Rishant Jung Karki

Student ID: 2408591

Module: Artificial Intelligence & Machine learning

Module Leader: Mr. Siman Giri

Tutor: Ms. Sunita Parajuli

Contents

Part II: Image Classification Report	
2 Abstract.....	1
1. Introduction.....	1
2. Dataset	2
3. Methodology.....	2
3.1.1 Baseline CNN Model.....	2
3.1.2 Deeper CNN with Regularisation	3
3.1.3 Optimizer Strategy	4
3.2 Part B: Transfer Learning	4
4. Experiments and Results.....	5
4.1 Baseline CNN Model Results.....	5
4.2 Deeper CNN — Adam vs SGD Comparison	5
4.4 Training Curves and Confusion Matrices.....	5
5. Fine-Tuning and Transfer Learning Results.....	6
5.1 Feature Extraction Phase	6
5.2 Fine-Tuning Phase	6
5.3 Transfer Learning vs. From-Scratch Models.....	6
6. Conclusion and Future Work.....	7

Abstract

This report presents a comprehensive end-to-end deep learning study on image classification using Convolutional Neural Networks (CNNs). The primary objective is to investigate the effectiveness of different CNN architectures and training strategies on the Rice Leaf Disease dataset, a dataset comprising images across 6 distinct classes: Bacterial Leaf Blight, Leaf Blast, Sheath Blight, Brown Spot, Leaf Scald, and Healthy Rice Leaf. The study is divided into two main parts. In Part A, a baseline CNN model is designed and trained from scratch using three convolutional blocks with filters of size 32, 64, and 128, followed by a deeper regularised architecture incorporating Batch Normalisation and Dropout layers with four convolutional blocks (32, 64, 128, 256 filters). Both architectures are evaluated and compared across key performance metrics including accuracy, precision, recall, and F1-score, as well as different optimisers (SGD and Adam). In Part B, transfer learning is applied by fine-tuning a pre-trained VGG16 model to leverage feature representations learned on ImageNet. Experimental results demonstrate that the fine-tuned VGG16 transfer learning model outperformed all from-scratch models, while the deeper architecture with Adam outperformed the baseline. The findings highlight the trade-offs between model complexity, computational cost, and classification performance, providing practical insights into building effective image classification systems.

1. Introduction

Image Classification is considered the most fundamental topic in Computer Vision and Artificial Intelligence research field. Automatic recognition and classification of visual information have several applications in different fields including healthcare, farming, automobiles, e-commerce, as well as security. Over the past few years, Deep Learning algorithms, specifically the Convolutional Neural Networks (CNN), have become the most accurate and important approach for performing Image Classification.

Convolutional Neural Networks (CNN) are tailored networks that are mainly designed for dealing with data in structured grids like Images. The design of CNNs employs the use of convolution, pooling, and local receptive fields in order to analyze the hierarchical structure of data that cannot be captured by the traditional Feed-Forward Neural Networks. This method allows them to learn both low-level properties, such as edges and texture, as well as high-level properties like shapes and objects while utilizing very few parameters.

Although CNN has succeeded, the successful training of CNNs from scratch is a difficult endeavor. As the depth of the CNN increases and its representational ability improves, there is a risk that the model might overfit, particularly when training data is limited. To address this problem, CNN regularization techniques have been introduced, including Dropout and Batch Normalization. These regularization techniques play a crucial role in the current CNN model design. In addition, the transfer learning technique proves to be effective in making use of models pretrained using large datasets, such as ImageNet, to solve domain-specific tasks requiring fewer resources.

In this study, the Rice Leaf Disease dataset will serve as the target dataset. There are two goals in the proposed experiment, namely, (1) developing, training, and carefully analyzing CNN models designed from scratch and (2) implementing transfer learning using a pre-trained VGG16 model.

In addition to the Introduction part, the structure of this report also includes the following sections: The second section outlines the dataset while the third section talks about the

methodology followed and model architecture. Section 5 covers the transfer learning approach; and finally Section 6 concludes with key findings and suggestions for future work.

2. Dataset

In this project, the dataset used is Rice Leaf Disease dataset (Rice_Leaf_AUG). It is a dataset that consists of labelled images of augmented rice leaves that have been obtained from a local directory.

This particular dataset contains images classified in 6 different categories. The different categories are Bacterial Leaf Blight, Leaf Blast, Sheath Blight, Brown Spot, Leaf Scald, and Healthy Rice Leaf. All the images are balanced with regards to their class. The images are RGB type and are stored in JPEG format.

The dataset has been divided into a training set and a testing set in an 80% and 20% ratio respectively using the stratified sampling technique.

The image pre-processing techniques used before training the models are as follows:

- Resize: Resizing all images to 150 x 150 (224 x 224 in the case of the VGG16 Transfer Learning Model) to ensure that the image input is consistent.
- Normalization: Normalization of the pixel values by scaling from 0 to 1 by dividing by 255.
- Data Augmentation: Involves random horizontal flipping, rotation range of 40 degrees, brightness of between 0.85-1.15, small shift in width and height of 0.002, and shear of 12.5 degrees. No vertical flips and zoom were performed. The fill mode is 'nearest'.

Image data generator from keras is used to perform image pre-processing.

3. Methodology

The part of this report describes the models of deep learning which have been designed for the process of image classification. This methodology part of the report consists of two parts. The first part deals with designing the CNN models while the second part deals with developing the model by transferring the learning. All the models described above have been developed using Keras deep learning library with Tensorflow.

3.1 Part A: CNN Models from Scratch

3.1.1 Baseline CNN Model

The baseline architecture comprises a basic CNN architecture with three convolutional blocks, each made up of a convolutional layer with a max-pooling layer after it, followed by three dense layers and a softmax output layer. This design was deliberately kept simple to serve as a benchmark against which the more sophisticated models will be compared.

A convolutional layer involves applying a series of filters that are learned from training the data to detect spatially localized features of the image. Relu activations are included in the design to inject non-linear properties. After each convolutional block, max-pooling layers with 3×3 pool sizes are added to downsample the feature maps' spatial dimensions. The output feature maps are then flattened into a one-dimensional vector to feed into dense layers, which then lead to a softmax output layer with six neurons, one for each class label.

Key hyperparameters for the baseline model are as follows:

Hyperparameter	Value
Input Image Size	$150 \times 150 \times 3$
Conv Layer Filters	32, 64, 128
Kernel Size	4×4
Activation (Conv)	ReLU
Pooling Size	3×3 (MaxPooling)
Dense Layers	$128 \rightarrow 64 \rightarrow 32 \rightarrow 6$ (softmax)
Optimiser	Adam ($\text{lr}=0.001$, $\beta_1=0.869$, $\beta_2=0.995$)
Loss Function	Categorical Cross-Entropy
Batch Size	32
Epochs	Up to 40 (EarlyStopping patience=8)

3.1.2 Deeper CNN with Regularisation

In order to study the impact of increasing the model complexity and implementing regularization, a more complex CNN model was built with four Convolutional Blocks — roughly doubling the amount of Convolutional Layers from the basic model. For regularization, two methods were implemented:

- Batch Normalization: Introduced after each Convolutional Layer in order to normalise layer output, stabilize training process, and increase possible learning rate value. The method helps prevent internal covariate shift; hence, accelerating the speed of convergence and performing as an additional weak regularizer.
- Dropout: Implemented after each Convolutional Block (probabilities 0.25, 0.25, 0.3, 0.3) and after the Dense layer (probability 0.5).

The deeper model architecture is as follows:

Block	Layers	Filters	Regularisation
Block 1	Conv2D $\times 2$ + MaxPool	32	BatchNorm + Dropout(0.25)

Block 2	Conv2D $\times$ 2 + MaxPool	64	BatchNorm + Dropout(0.25)
Block 3	Conv2D $\times$ 2 + MaxPool	128	BatchNorm + Dropout(0.3)
Block 4	Conv2D $\times$ 2 + MaxPool	256	BatchNorm + Dropout(0.3)
Dense	512 $\rightarrow$ 6 (softmax)	—	BatchNorm + Dropout(0.5)

3.1.3 Optimizer Strategy

In order to examine how the selection of the optimizer affects the model's convergence process and its end result, the model with deeper CNN was trained based on two optimizers' settings:

- Stochastic Gradient Descent (SGD): It is an old-fashioned first-order optimizer that updates its weights based on the gradient calculation for each mini-batch. SGD uses learning rate = 0.01 and momentum = 0.9. This type of optimizer generalizes relatively well, although, it might take more time to converge in comparison with other optimizers.
- Adam (Adaptive Moment Estimation): It is an adaptive optimizer that uses some of the advantages of the momentum and RMSProp. Adam optimizer is set with learning rate = 0.001. This optimizer converges rather quickly in comparison with SGD.

Both optimizers were tested with categorical cross-entropy, which is suitable for multiclass classification tasks. The performance of both optimizers is being compared by evaluating their convergence process and final testing accuracy.

3.2 Part B: Transfer Learning

The technique utilizes the knowledge contained within the models trained on big data — in this scenario, ImageNet — for a new classification problem. This process is especially effective when there is only a smaller amount of data available, since the pre-trained model will have already developed visual features that can be hard to learn on a small dataset.

In this particular experiment, the selected pre-trained model is the VGG16. The VGG16 model suits well for medium-sized datasets and is an efficient compromise in terms of complexity and speed. It is a popular architecture and is characterized by its simple structure that comprises 13 convolutional layers with fully connected layers after it. The convolutional part of the network was loaded with weights of ImageNet, and the head consisting of classification of 1000 classes was stripped off.

The two approaches for training the models are listed below:

- Feature Extraction: During the first stage, all the layers of the pre-trained model remained frozen, meaning that the parameters of such layers could not be altered during the training procedure. The training process involved only the newly introduced classification layers (Flatten $\rightarrow$ Dense(256, ReLU) $\rightarrow$ Dense(6, softmax)).

- Fine-Tuning: During the second stage, the layers starting from the 15th one and higher became unfrozen and able to update during training by applying a low learning rate of $1e-5$. Also, an augmentation `RandomFlip('horizontal')` was applied. Thus, high-level feature extractors will adapt to the specifics of the dataset while preserving the knowledge gained on lower levels.

Input data were scaled to 224×224 pixels, which is the size of input images for the VGG16 model. The models were trained using Adam optimiser with learning rates corresponding to those listed above and categorical cross-entropy loss function.

4. Experiments and Results

These results are from experiments carried out on all Part A models: baseline CNN model and deep CNN model using Adam and SGD optimizers. Experiments were conducted on all these models using the test set (which is 20% of the Rice Leaf Disease dataset) where evaluation was done using accuracy, precision, recall, and F1-score.

4.1 Baseline CNN Model Results

The baseline CNN model was developed using the Adam optimiser (learning rate=0.001) for a maximum of 40 epochs with early stopping. This model achieved convergence at a satisfactory level, acting as the baseline for comparison. The model exhibited good classification capability but appeared somewhat constrained in its ability to learn discriminative features within visually similar diseases.

4.2 Deeper CNN — Adam vs SGD Comparison

The deeper CNN was trained using Adam optimizer (lr=0.001) and SGD optimizer (lr=0.01, momentum=0.9) for a maximum of 50 epochs. The deeper CNN trained using Adam achieved faster convergence with higher validation accuracy when compared to the one trained using SGD.

The main takeaways from the optimizer experiments:

- Adam optimizer provided faster convergence as compared to SGD optimizer. • The loss with the SGD optimizer showed a slower descent but was relatively responsive to learning rate scheduling with the help of `ReduceLROnPlateau` callback.
- The Adam-deeper CNN performed better than the baseline CNN. This indicates that adding more convolution layers to the network with batch normalization and dropout helps improve classification accuracy.
- The performance of both variations of the deeper CNN surpassed the baseline, indicating the effectiveness of the applied regularization techniques.

4.4 Training Curves and Confusion Matrices

Accuracy/loss curves were obtained for each model. It was clear that the training curve of the base model diverged from the validation curve as training progressed, suggesting some degree of overfitting. The deeper model with dropout and batch normalization had tighter curves, which suggested successful regularization of the network. As far as optimization methods are concerned, the deeper network took longer to converge with SGD than with Adam.

The confusion matrices of all models have been obtained using the test data. The baseline network gave the worst results, especially when discriminating visually similar class pairs like Leaf Blast vs Brown Spot. However, the deeper network using Adam yielded lower inter-class

confusion rates. ROC curves for the deeper network using both Adam and SGD have also been produced, with the former yielding higher AUC scores for all six classes.

The analysis of the unknown data confirmed these results: the baseline model performed poorly on unknown data while the deeper one (Adam) yielded slightly better results. The fine-tuned transfer learning model yielded the best results on unknown rice leaf images.

5. Fine-Tuning and Transfer Learning Results

The transfer learning model using the VGG16 network pretrained on the ImageNet dataset showed superior results compared to the scratch training model. The pre-trained convolutional base of the VGG16 network was imported into the model with frozen layers, and a classifier layer was appended.

5.1 Feature Extraction Phase

In the feature extraction step, the pre-trained convolutional base VGG16 architecture was frozen, and the only part being trained was the classifier layer (Dense(256, ReLU) -> Dense(6, softmax)). This neural network was trained with Adam optimiser and categorical cross-entropy for 15 epochs. Even though there is no pre-training in task-specific convolutional layers, the pre-trained VGG16 features performed competitively well in comparison with the from-scratch networks.

5.2 Fine-Tuning Phase

In the fine-tuning phase, VGG16 layers from position 15 onwards were unfrozen and the full model was retrained with a very low learning rate of 1e-5 (Adam optimiser) for 25 epochs. A RandomFlip('horizontal') augmentation layer was incorporated at the input. The reduced learning rate prevented catastrophic forgetting of the base convolutional representations while allowing the high-level layers to adapt to the specific visual characteristics of rice leaf diseases.

The fine-tuned model demonstrated the following advantages over the feature extraction model:

- Improved accuracy on the test set due to task-specific adaptation of higher-level convolutional features.
- Better handling of visually similar disease classes through refined feature discrimination.
- Reduced misclassification on unseen data, demonstrating stronger generalisation capability.

5.3 Transfer Learning vs. From-Scratch Models

Both the frozen and fine-tuned VGG16 models performed better compared to other models trained from scratch based on test accuracy and generalization. Notable observations include:

- The fine-tuned VGG16 yielded the best performance among all models used for comparison, showing that transfer learning of features learnt from ImageNet dataset gives higher accuracy compared to training from scratch.
- Transfer learning takes much shorter training time to give similar or even better results compared to deeper CNN networks trained from scratch.

- The VGG16 model was able to utilize low-level feature representation learned in ImageNet, such as edges, textures, and colours that are useful in discriminating diseases of the rice leaf images.
- The fine-tuned VGG16 model gave the best performance based on unseen data tests and produced least classification errors in all six diseases of rice leaf categories.

6. Conclusion and Future Work

This research provided an analysis of the various CNN models used for image classification on the Rice Leaf Disease data set. In total, four different CNN variations were tested: the basic CNN architecture, deeper CNN with regularization training using both the SGD and Adam algorithms, and finally the transfer learning method using the VGG16 neural network. Some of the main conclusions drawn from the research include the following:

- The basic CNN provided the necessary reference point but had a constrained representation power and tended to overfit without regularization.
- Regularization using Batch Normalization and Dropout provided consistent improvement over the basic CNN model.
- The Adam optimizer outperformed SGD in convergence speed and final accuracy for the deeper CNN, making it the preferred choice for this classification task. SGD remained competitive in terms of generalization but required more training epochs.
- Transfer learning via VGG16 — both frozen and fine-tuned — outperformed all from-scratch models. The fine-tuned VGG16 achieved the best overall performance, demonstrating that adapting pre-trained ImageNet features to the target domain is highly effective.
- Unseen data evaluation confirmed the ranking: Fine-tuned VGG16 > Deeper CNN (Adam) > Deeper CNN (SGD) > Baseline CNN.

From the tradeoffs noticed above for model complexity, training time, and performance, there is one fundamental rule which comes out clearly: increased model complexity works only if there is adequate regularization to go along with it; and using pre-trained models is highly recommended as well.

Several directions for future work are proposed:

- Investigation of other up-to-date pre-trained architectures, such as ResNet50, EfficientNet, or MobileNetV3, as replacements for VGG16, which might have better efficiency-accuracy balances.
- The use of state-of-the-art data augmentation techniques, including CutMix, MixUp, and testtime augmentation, to enhance the generalizability of the model.
- Application of learning rate warm-up and cosine annealing techniques instead of using ReduceLROnPlateau to facilitate optimization.
- Further diversification of the training data by incorporating other classes of rice disease samples or other crop diseases for more comprehensive classification purposes.
- Distribution of the fine-tuned neural network in the form of a compact version that could run on a mobile device (such as TensorFlow Lite).

Code:

<https://github.com/Rishant-Jung-Karki/AI-ML-Assessment>